VECTOR

MICROSAR CRYPTO

Technical Reference

CRYPTO LIBCV

Version 4.2.0

Authors	Markus Schneider, Matthias Weniger
Status	Released

Document Information

History

Author	Date	Version	Remarks
Schneider, Markus	2017-03-07	1.01.00	Initial creation of Technical Reference
Weniger, Matthias	2017-08-31	2.01.00	Rework Document Updated supported features 3.1 Added chapter 3.1.3.3 Added chapter 3.1.3.4 Added chapter 3.6.2 Added chapter 3.7 Added chapter 3.10.1 Added description of all used EXCLUSIVE AREAS in chapter 4.2 Updated for behavior changes 5.2.9
Weniger, Matthias	2017-09-27	3.00.00	Updated supported features 3.1 Added chapter 3.9.1 Added chapter 3.9.2 Updated chapter 3.4 for SHE Master Key support Updated chapter 4.1.1 for new file
Weniger, Matthias	2017-10-27	3.01.00	Updated chapter 5 Added chapter 3.9.3 Added chapter 3.10.2 Added chapter 3.10.3 Grouped Key Usage 3.9 Grouped Algorithm 3.9.5
Weniger, Matthias	2017-11-23	3.02.00	Updated supported features in 3.1 Added chapter 3.8 Updated chapter 3.10.2
Weniger, Matthias	2017-12-19	3.03.00	Updated supported features and deviations 3.1 Update API Description for Key Derive Update chapter 3.7 Added chapter 3.7.1, 3.7.2 and 3.7.3
Weniger, Matthias	2018-01-19	3.04.00	Updated supported features and deviations 3.1 Updated text for extended information’s and descriptions 3.4 and 3.10.1 Updated chapter 3.9.1 and 7.2 Add Access Rights to 3.9.2 Added chapter 3.9.4, 3.11 and 4.3

Author	Date	Version	Remarks
Weniger, Matthias	2018-02-13	3.05.00	Resolved ESCAN00097738 with adding chapter 3.12.1 Updated supported features and deviations 3.1 Added chapter 3.9.5, 3.9.6, 3.10.4, 3.10.5 and 3.12 Rework chapter 3.10.1 for new behavior Updated chapter 3.8, 3.9.1, 3.9.2 and 3.10.2
Weniger, Matthias	2018-02-27	3.06.00	Updated supported features and deviations 3.1 Rework chapter 3.9.5, 3.10.4, 3.10.5 and 4.2
Weniger, Matthias	2018-05-30	4.00.00	Updated supported features and deviations 3.1 Rework chapter 3.1.1 and 5 Add chapter 3.10.6
Weniger, Matthias	2018-06-12	4.01.00	Updated supported features, deviations and limitations 3.1 Rework chapter 3.1.1, 3.4, 3.9.1, 3.9.2 and 3.9.5
Weniger, Matthias	2018-08-22	4.02.00	Update architecture 2.1 Rework chapter 3.4, 3.12.1, 4.1.1 and 0 Add chapter 5.1.3

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_CryptoDriver.pdf	4.3.0
[2]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	4.3.0
[4]	HIS	2009-04-01 SHE Functional Specification v1.1 (rev439)	1.1
[5]	NIST	NIST Special Publication 800-108	2009-10
[6]	NIST	FIPS PUB 186-4	2013-07
[7]	NIST	NIST Special Publication 800-56A	2
[8]	ISO	ISO 15118-2	2014-04-01
[9]	NIST	FIPS186-2	2000-01
[10]	NIST	SP800-90A	1

Contents

1 Component History 10

2 Introduction 11
2.1 Architecture Overview 11

3 Functional Description 13
3.1 Features 13
3.1.1 Deviations 13
3.1.2 Additions/ Extensions 14
3.1.3 Limitations 14
3.1.3.1 Cryptographic Algorithms and Modes 14
3.1.3.2 Certificate Handling 14
3.1.3.3 Key Generation 14
3.1.3.4 AEAD Det Checks 14
3.1.3.5 Key Derivation according to ISO15118 14
3.1.3.6 ProcessJob length parameter 14
3.1.3.7 Key Length 14
3.2 Initialization 15
3.3 Main Functions 15
3.4 SHE Key Update Protocol 15
3.4.1 Key Flags 16
3.4.2 Preconditions 18
3.5 Implementation of esl_getBytesRNG 19
3.6 Error Handling 19
3.6.1 Development Error Reporting 19
3.6.2 Algorithm Parameter Overview 21
3.7 Key Derivation 22
3.7.1 Symmetric and Asymmetric Key Derivation 22
3.7.2 Concatenation KDF 23
3.7.2.1 Preconditions 23
3.7.3 Smart Charge Communication (SCC) 23
3.7.3.1 Preconditions 23
3.7.3.2 Certificate Installation 23
3.7.3.3 Certificate Update 23
3.8 Key Exchange 24
3.9 Key Usage 24
3.9.1 Custom Key Elements 25
3.9.2 Key Element Access Rights 25
3.9.3 Key Locking 26

3.9.4 Pre-Configured Key Types ... 26
3.9.5 Key Validity ... 27
3.9.6 Key Element Delete ... 28
3.10 Algorithm ... 28
3.10.1 AES CMAC Round-key Reuse ... 28
3.10.2 RSA ... 29
3.10.3 ECDSA ... 30
3.10.4 RNG FIPS ... 30
3.10.5 RNG DRBG ... 30
3.10.6 AES Encryption/Decryption ... 32
3.11 Driver Objects and Jobs ... 32
3.12 Memory Access ... 33
3.12.1 NVM Support ... 33
3.12.2 vStdLib ... 33

4 Integration ... 34
4.1 Scope of Delivery ... 34
4.1.1 Static Files ... 34
4.1.2 Dynamic Files ... 34
4.2 Critical Sections ... 36
4.3 Best Practice ... 36
4.3.1 Bottom to Top ... 36

5 API Description ... 37
5.1 Services provided by CRYPTO ... 37
5.1.1 Crypto_30_LibCv_Init ... 37
5.1.2 Crypto_30_LibCv_InitMemory ... 38
5.1.3 Crypto_30_LibCv_MainFunction ... 38
5.1.4 Crypto_30_LibCv_GetVersionInfo ... 39
5.1.5 Crypto_30_LibCv_ProcessJob ... 39
5.1.6 Crypto_30_LibCv_CancelJob ... 41
5.2 Key Management Functions ... 42
5.2.1 Crypto_30_LibCv_KeyCopy ... 42
5.2.2 Crypto_30_LibCv_KeyElementCopy ... 43
5.2.3 Crypto_30_LibCv_KeyElementIdsGet ... 44
5.2.4 Crypto_30_LibCv_KeyElementSet ... 45
5.2.5 Crypto_30_LibCv_KeyValidSet ... 46
5.2.6 Crypto_30_LibCv_KeyElementGet ... 47
5.2.7 Crypto_30_LibCv_RandomSeed ... 48
5.2.8 Crypto_30_LibCv_KeyGenerate ... 49
5.2.9 Crypto_30_LibCv_KeyDerive ... 50

5.2.10 Crypto_30_LibCv_KeyExchangeCalcPubVal 51
5.2.11 Crypto_30_LibCv_KeyExchangeCalcSecret 52
5.2.12 Crypto_30_LibCv_CertificateParse 53
5.2.13 Crypto_30_LibCv_CertificateVerify 54
5.3 Services needed by CRYPTO 55
5.3.1 esl_getBytesRNG 55
5.4 Services used by CRYPTO 56

6 Configuration 57
6.1 Configuration Variants 57

7 Glossary and Abbreviations 58
7.1 Glossary 58
7.2 Abbreviations 58

8 Contact 60

Illustrations

Figure 2-1 AUTOSAR 4.2 Architecture Overview 11
Figure 2-2 Interfaces to adjacent modules of the CRYPTO 12

Tables

Table 1-1 Component history 10
Table 3-1 Supported AUTOSAR standard conform features 13
Table 3-2 Not supported AUTOSAR standard conform features 13
Table 3-3 Features provided beyond the AUTOSAR standard 14
Table 3-4 SHE Key Mapping 15
Table 3-5 SHE Key Update Rights 16
Table 3-6 SHE Key Service usage 16
Table 3-7 FID usage of SHE keys 17
Table 3-8 FID usage of Services 18
Table 3-9 Service IDs 20
Table 3-10 Errors reported to DET 20
Table 3-11 Overview of the required algorithm parameter 21
Table 3-12 Key Derivation Functions 22
Table 3-13 Values for Key Derivation Algorithm Key Element 22
Table 3-14 Key Exchange Algorithm 24
Table 3-15 Custom Key Elements 25
Table 3-16 Key Element Access Rights Ranking 26
Table 3-17 Key Element Access Rights 26
Table 3-18 Key Validity 27
Table 3-19 RSA Key Element Usage 29
Table 3-20 ECDSA Signature Key Element Storage 30
Table 3-21 ECDSA Signature Storage 30
Table 3-22 DRBG required key elements 31
Table 4-1 Static files 34
Table 4-2 Generated files 34
Table 5-1 Crypto_30_LibCv_Init 37
Table 5-2 Crypto_30_LibCv_InitMemory 38
Table 5-3 Crypto_30_LibCv_MainFunction 38
Table 5-4 Crypto_30_LibCv_GetVersionInfo 39
Table 5-5 Crypto_30_LibCv_ProcessJob 40
Table 5-6 Crypto_30_LibCv_CancelJob 41
Table 5-7 Crypto_30_LibCv_KeyCopy 42
Table 5-8 Crypto_30_LibCv_KeyElementCopy 43
Table 5-9 Crypto_30_LibCv_KeyElementIdsGet 44
Table 5-10 Crypto_30_LibCv_KeyElementSet 45
Table 5-11 Crypto_30_LibCv_KeyValidSet 46
Table 5-12 Crypto_30_LibCv_KeyElementGet 47
Table 5-13 Crypto_30_LibCv_RandomSeed 48
Table 5-14 Crypto_30_LibCv_KeyGenerate 49
Table 5-15 Crypto_30_LibCv_KeyDerive 50
Table 5-16 Crypto_30_LibCv_KeyExchangeCalcPubVal 51
Table 5-17 Crypto_30_LibCv_KeyExchangeCalcSecret 52
Table 5-18 Crypto_30_LibCv_CertificateParse 53
Table 5-19 Crypto_30_LibCv_CertificateVerify 54
Table 5-20 esl_getBytesRNG 55
Table 5-21 Services used by the CRYPTO 56
Table 7-1 Glossary 58

Table 7-2 Abbreviations 59

© 2018 Vector Informatik GmbH

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00	Initial beta release
1.01	Adaptions to the specification; several improvements and bugfixes
2.01	Safe BSW; Support of reuse of round-keys for AES CMAC; several improvements and bugfixes
3.00	Support of RSA; Support update with SHE Master Key; Check Key Access Rights
3.01	Support of ECDSA for NIST, ANSI and SEC with P256R1 and SHA-256 pre-Hashing; Key Locking
3.02	Support ECDHE with ANSIp256r1, HMAC-SHA-1 and HMAC-SHA-256
3.03	Support concatenation KDF (NIST 800-56A) and certificate installation and update according to ISO15118
3.04	Support ECDSA for ANSIp256r1 with SHA-1 pre-Hashing
3.05	Support CTR-DRBG using AES-128 according to NIST SP 800-90A, Support RSA CRT DSA SHA-1 and SHA-256 and PKCS1_V15, Support ECDHE for SECp256r1, Support deletion of key elements
3.06	Support AES 128 encryption and decryption without padding
4.00	Support AES 256 encryption and decryption, Support of ECDSA for NIST and SEC with P384R1
4.01	SHE Key Update Protocol - Support UID, FID and counter
4.02	Call NvM_SetRamBlockStatus

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CRYPTO as specified in [1].

Supported AUTOSAR Release*:	4.3	
Supported Configuration Variants:	pre-compile	
Vendor ID:	CRYPTO_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CRYPTO_MODULE_ID	114 decimal (according to ref. [3])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The Crypto Driver (CRYPTO) is called by the Crypto Interface (CRYIF) and performs the specific cryptographic functionality. The CRYPTO specification [1] offers a superset of algorithms which can be extended by ‘custom algorithms’. This software-based Crypto Driver offers a subset of algorithms and features which is described in 3.1.

2.1 Architecture Overview

The following figure shows where the CRYPTO is located in the AUTOSAR architecture.

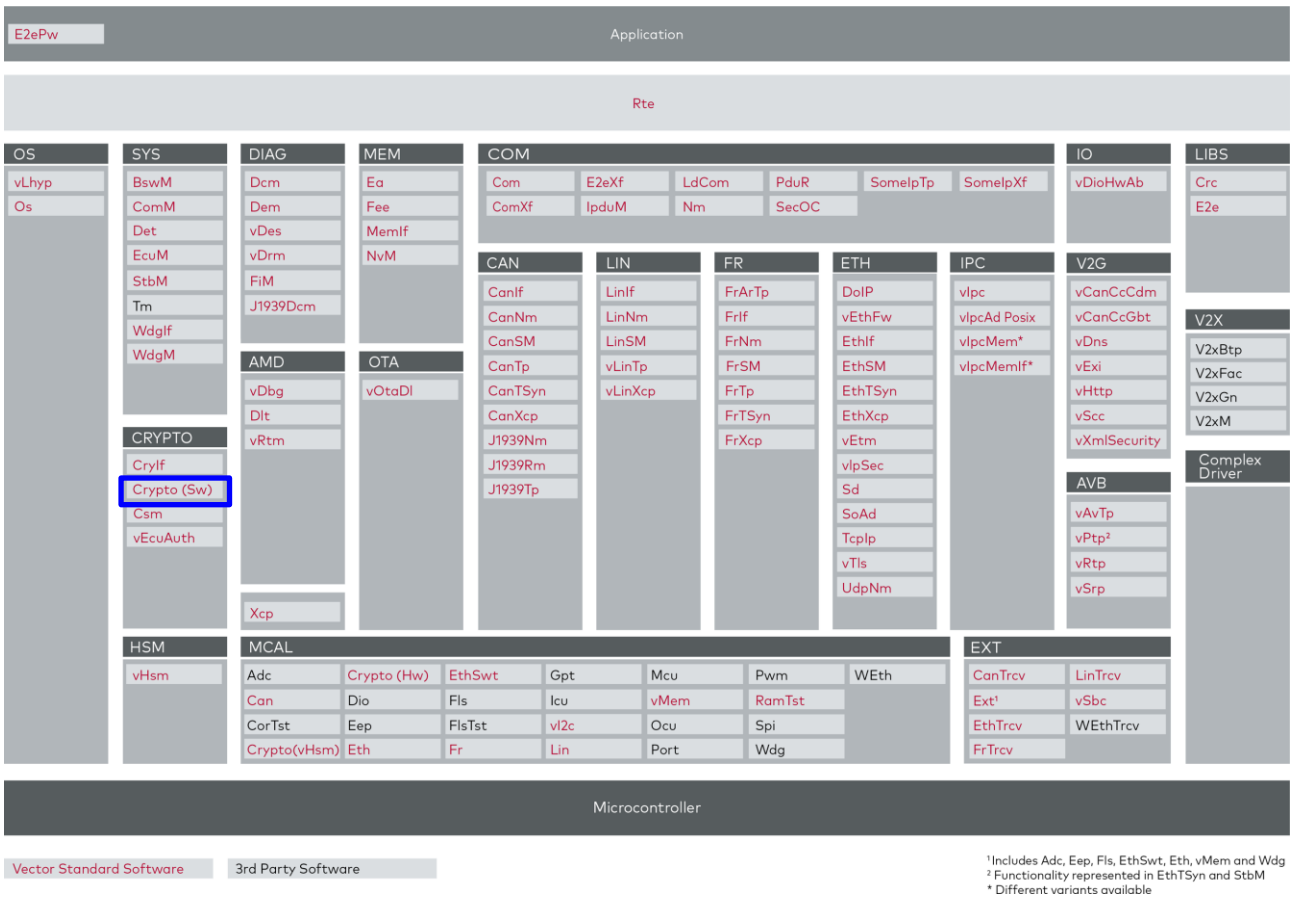


Figure 2-1 AUTOSAR 4.2 Architecture Overview

The next figure shows the interfaces to adjacent modules of the CRYPTO. These interfaces are described in chapter 5.

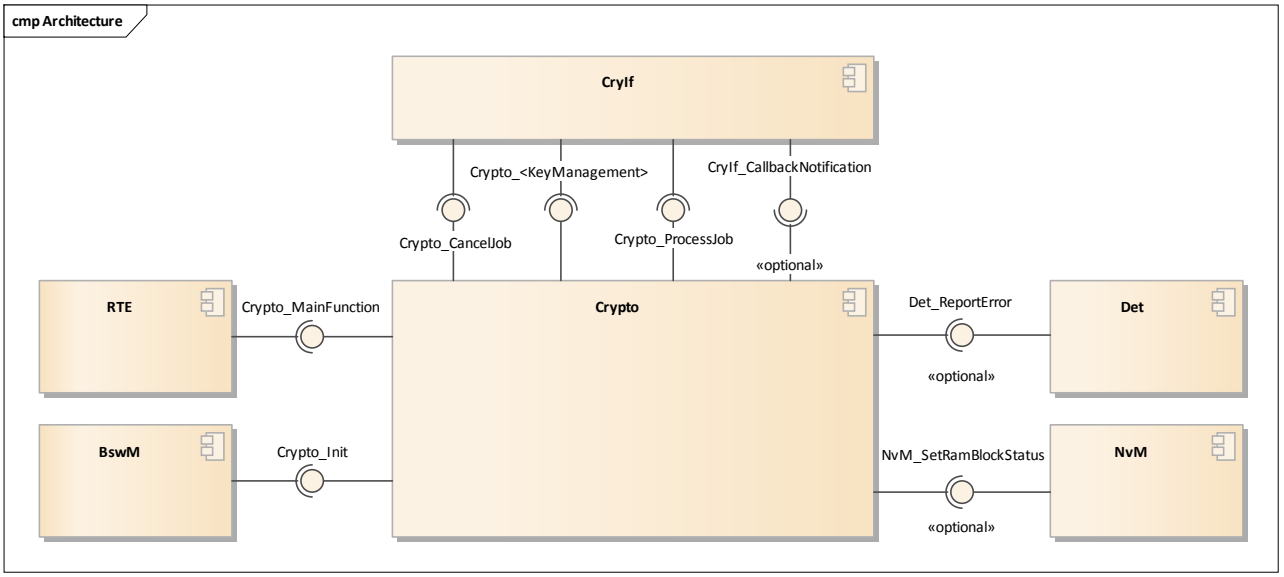


Figure 2-2 Interfaces to adjacent modules of the CRYPTO

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the CRYPTO.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further CRYPTO functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-3 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
AEAD: AES-GCM
Encrypt/Decrypt: AES-128 and AES-256 ECB/CBC with PKCS7 and without padding; RSA PKCS1-V1.5
HASH: SHA-1; SHA-256; SHA-512
MAC: AES CMAC; SipHash, HMAC-SHA-1, HMAC-SHA-256
PRNG: FIPS 186-2, DRBG-AES128 with DF and without DF
Signature: Ed25519; RSA PKCS1-V1.5, PKCS1-V1.5 CRT and PSS with SHA-1/SHA-256; NIST/ANSI/SECp256r1 without pre-hashing, SHA1 and SHA256 pre-hashing; NIST/SECp384r1 without pre-hashing, SHA1 and SHA256 pre-hashing
Key Derive: KDF in Counter Mode, KDF in Counter Mode with Appendix, concatenation KDF (NIST 800-56A) and certificate installation and update according to ISO15118
Key Exchange: ECDHE with ANSIp256r1, SECp256r1 and X25519

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
Certificate handling
Symmetric and asymmetric key generation
All algorithms not stated in Table 3-1
The access rights are mapped to different values according to 3.9.2

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond the AUTOSAR Standard
Multiple CryptoDriverObjects to support multiple workspaces per service
Crypto_KeyElementSet supports an adapted variant of the ‘SHE Key Update Protocol’ [4]
The AES CMAC Supports Round-key Reuse
Use vStdLib for memory access
Support deletion of key elements

Table 3-3 Features provided beyond the AUTOSAR standard

3.1.3 Limitations

3.1.3.1 Cryptographic Algorithms and Modes

Only a subset of the stated algorithms and modes in the AUTOSAR SWS [1] are currently supported. The list of algorithms is described in Table 3-1.

3.1.3.2 Certificate Handling

Currently there is no implementation to parse and verify certificates. However, the API is still available for compatibility reasons.

3.1.3.3 Key Generation

Currently there is no implementation to generate keys. However, the API is still available for compatibility reasons.

3.1.3.4 AEAD Det Checks

The Det checks for AEAD differ from the table in [1]. There is a different handling required for the AEAD implementation. The parameter check is described in 3.6.2.

3.1.3.5 Key Derivation according to ISO15118

The ISO15118 certificate handling requires a validity check of the new ECC key pair specified in ISO15118 [V2G2-823]. This check is currently not part of the implementation. To be complied to IS15118 the validity of the ECC key pair need to be checked by the caller. To check if the ECC key pair is valid, the new key pair could be used for generating and verifying a signature.

3.1.3.6 ProcessJob length parameter

The max length which can be handled by one ProcessJob call is 65535 (uint16 max value), event the length type is designed as uint32 in [1]. To process longer data, use the streaming approach with multiple update.

3.1.3.7 Key Length

Even if the length of the key is designed as uint32 in [1], the max key length is restricted to a value of 65535 (uint16 max value) due to internal limitations.

3.2 Initialization

Before any other functionality of the CRYPTO module can be called the initialization function `Crypto_30_LibCv_Init()` has to be called by the BSWM.

For manual null initialization of RAM variables, the CRYPTO offers the function `Crypto_30_LibCv_InitMemory()` which can be called before the `Crypto_30_LibCv_Init()`.

3.3 Main Functions

The CRYPTO module implementation provides one main function. When the usage of asynchronous job processing is enabled, this main function has to be called cyclically on task level. The main function is responsible to start processing of the job.

3.4 SHE Key Update Protocol

The MICROSAR CRYPTO has the ability to update a key element by using a simplified SHE Key Update Protocol [4]. The simplified Protocol does not support all SHE features. The keys will be hold as plaintext in the CRYPTO. The NVM can be encrypted by user. The SHE update implementation allows up to 255 SHE pages with up to 10 User and 1 RAM key per page. Secret, Master and Boot and Boot Mac may only exist in the Page 0, which is illustrated in the following table.

Mapping SHE Key	SHE Id	Page
SECRET_KEY	0	0
MASTER_ECU_KEY	1	0
BOOT_MAC_KEY	2	0
BOOT_MAC	3	0
KEY_1_PAGE_M	4	M
KEY_2_PAGE_M	5	M
KEY_3_PAGE_M	6	M
KEY_4_PAGE_M	7	M
KEY_5_PAGE_M	8	M
KEY_6_PAGE_M	9	M
KEY_7_PAGE_M	10	M
KEY_8_PAGE_M	11	M
KEY_9_PAGE_M	12	M
KEY_10_PAGE_M	13	M
RAM_KEY_PAGE_M	14	M

Table 3-4 SHE Key Mapping

Before updating a SHE key, the corresponding update rights must be checked. The key rights depend on the used key and the used service. The dependencies are shown in Table 3-5 and Table 3-6. To update a RAM_KEY as plaintext will not be supported.

Authentication Key / Update Key	SECRET_KEY	MASTER_ECU_KEY	BOOT_MAC_KEY	BOOT_MAC	KEY_{n}	RAM_KEY_{Page}
SECRET_KEY						
MASTER_ECU_KEY		X				
BOOT_MAC_KEY		X	X			
BOOT_MAC		X	X			
KEY_{n}		X			X	
RAM_KEY_{Page}	X				X*	

X* only keys on the same SHE page

Table 3-5 SHE Key Update Rights

Service / SHE Key	SECRET_KEY	MASTER_ECU_KEY	BOOT_MAC_KEY	BOOT_MAC	KEY_{n}	RAM_KEY_{Page}
Decrypt					X**	X
Encrypt					X**	X
Mac Generate					X**	X
Mac Verify			X		X**	X
KeyElementSet		X**	X**	X**	X**	X
Authentication Key	X	X	X		X	
KeyElementGet*	X*	X*	X*	X*	X*	X*

*Read Access Is Allowed / Key will be provided as plaintext key.
** Depends on the FID

Table 3-6 SHE Key Service usage

3.4.1 Key Flags

The Crypto the following key flags (FID). The initial value for the FID is 0.

> WRITE PROTECTION
If the flag is set to “1” the SHE key cannot be written anymore.

- BOOT PROTECTION
  If the flag is set to “1” the key cannot be used if the Boot process is not finished.
- DEBUGGER PROTECTION
  If the flag is set to “1” the key cannot be used if a Debugger is attached.
- KEY USAGE
  If the flag is “1” the key can be used for CMAC otherwise for enc- and decryption.
- WILDCARD
  If the flag is “1” the wildcard cannot be used within M1 for SHE key update.
- CMAC USAGE
  This flag is only checked is KEY USAGE is “1” otherwise it shall be “0”, If the flag is set the key can only be used for CMAC verification. (Additional to the SHE standard)

The tables Table 3-7 and Table 3-8 show the usage of the key flags.

FID / SHE Key	WRITE PROTECTION	BOOT PROTECTION	DEBUGGER PROTECTION	KEY USAGE	WILDCARD	CMAC USAGE	Counter
SECRET_KEY		X*	X*				
MASTER_ECU_KEY	X	X	X		X		X
BOOT_MAC_KEY	X		X		X		X
BOOT_MAC	X		X		X		X
KEY_{n}	X	X	X	X	X	X	X
RAM_KEY_{Page}							
X* use flags from master key.							

Table 3-7 FID usage of SHE keys

The counter is not check for key usage, if the user set the key valid the key can be used.

FID / Service	WRITE PROTECTION	BOOT PROTECTION	DEBUGGER PROTECTION	KEY USAGE	WILDCARD	CMAC USAGE	Counter
Decrypt		X	X	X			
Encrypt		X	X	X			
Mac Generate		X	X	X		X	
Mac Verify		X	X	X		X	
KeyElementSet	X	X	X		X		X
KeyElementGet		X*	X*				

*Read Access Is Allowed

Table 3-8 FID usage of Services

3.4.2 Preconditions

When a key shall be updated using the SHE Key Update Protocol the

- WriteAccess has to be Encrypted.
- Key material must be a 64 byte long concatenated M1|M2|M3 message.
- The SHE key element need to be configured with the size of 16 bytes.
- If M4 and M5 are needed the “proof” key element must be of size 48 bytes. If the “proof” key element is not present, the SHE Key Update will be performed without calculating the proof.
- Enable check of FID.
  If the check is disabled all flags will be interpreted as 0.
- Enable check of Counter.
  If the check is disabled the counter will be ignored. If the counter is enabled the SHE key need the “CRYPTO_KE_CUSTOM_SHE_COUNTER” key element, otherwise it can be excluded from configuration. The “CRYPTO_KE_CUSTOM_SHE_COUNTER” key element must be of size 4 bytes with an initialization value.
- SHE Info Key.
  The SHE info key holds the information for UID, Debugger- and Boot-Protection.
  - UID
    If the UID is used the key element “CRYPTO_KE_CUSTOM_SHE_UID” must be configured with the size 15 bytes.
  - Debugger-Protection
    If the Debugger-Protection is used the key element “CRYPTO_KE_CUSTOM_SHE_DEBUGGER_PROTECTION” must be configured with the size 1 bytes. Otherwise it can be excluded from configuration. Setting this key element to 0x01u means that no debugger is attached and the keys with set Debugger-Protection will be accessible. Any other value will restrict the usage.
  - Boot-Protection
    If the Boot -Protection is used the key element “CRYPTO_KE_CUSTOM_SHE_BOOT_PROTECTION” must be configured with the size 1 bytes. Otherwise it can be excluded from configuration. Setting this key element to 0x01u means that the boot process is interpreted as passed and the keys with Boot-Protection will be accessible. Any other value will restrict the usage.



Caution
The keys with Debugger- and Boot-Protection will be hold in Ram but will not be usable if the access protection is activated. The access protection is never set by the CRYPTO itself, so corresponding key elements needs to be set from the integrator getting this information from the hardware.

SHE Keys cannot be copied or exported.



SHE Key are not validated automatically
SHE Keys with an initialization value are not set to valid during initialization like it is done for normal keys.

3.5 Implementation of esl_getBytesRNG

Currently the esl_getBytesRNG function is not provided within the Crypto_30_LibCv and the SecMod (Crypto Library) package.

Please implement this function within your project as specified in 5.3.1.



Expert Knowledge
By ensuring that initialization of the provided CRYPTO random number generator takes place before using elliptic curve operations, the esl_getBytesRNG implementation itself can call the configured Csm_RandomGenerate implementation.

3.6 Error Handling

3.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `CRYPTO_DEV_ERROR_REPORT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported CRYPTO ID is 114.The reported service IDs identify the services which are described in 5.1. The following table presents the service IDs and the related services:

Service ID	Service
0x00	Crypto_30_LibCv_Init()
0x01	Crypto_30_LibCv_GetVersionInfo()
0x03	Crypto_30_LibCv_ProcessJob()
0x0E	Crypto_30_LibCv_CancelJob()
0x04	Crypto_30_LibCv_KeyElementSet()
0x05	Crypto_30_LibCv_KeyValidSet()
0x06	Crypto_30_LibCv_KeyElementGet()
0x0F	Crypto_30_LibCv_KeyElementCopy()
0x10	Crypto_30_LibCv_KeyCopy()
0x11	Crypto_30_LibCv_KeyElementIdsGet()
0x0D	Crypto_30_LibCv_RandomSeed()
0x07	Crypto_30_LibCv_KeyGenerate()
0x08	Crypto_30_LibCv_KeyDerive()
0x09	Crypto_30_LibCv_KeyExchangeCalcPubVal()
0x0A	Crypto_30_LibCv_KeyExchangeCalcSecret()
0x0B	Crypto_30_LibCv_CertificateParse()
0x12	Crypto_30_LibCv_CertificateVerify()
0x0C	Crypto_30_LibCv_MainFunction()

Table 3-9 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x00	CRYPTO_E_UNINIT
0x01	CRYPTO_E_INIT_FAILED
0x02	CRYPTO_E_PARAM_POINTER
0x04	CRYPTO_E_PARAM_HANDLE
0x05	CRYPTO_E_PARAM_VALUE

Table 3-10 Errors reported to DET

3.6.2 Algorithm Parameter Overview

The required algorithm parameters are described in the following table. The required parameter differs from the table in [1].

Service \ Member	inputPtr	inputLength	secondaryInputPtr	secondaryInputLength	tertiaryInputPtr	tertiaryInputLength	outputPtr	outputLengthPtr	secondaryOutputPtr	secondaryOutputLengthPtr	verifyPtr	output64Ptr	mode
HASH	U	U					F	F					SUF
MACGENERATE	U	U					F	F					SUF
MACVERIFY	U	U	F	F							F		SUF
ENCRYPT	U	U					UF	UF					SUF
DECRYPT	U	U					UF	UF					SUF
AEADENCRYPT	U	U	V	U~			UF	UF	F	F			SUF
AEADDECRYPT	U	U	V	U~	F	F	UF	UF			F		SUF
SIGNATUREGENERATE	UF	U F~					F	F					SUF
SIGNATUREVERIFY	UF	U F~	F	F							F		SUF
SECCOUNTERINCREMENT													
SECCOUNTERREAD												F	
RANDOMGENERATE							F	F					

S: member required in Start mode.
U: member required in Update mode.
V: member optional in Update mode.
F: member required in Finish mode.
~: no Det check required / 0 is a valid value.

Table 3-11 Overview of the required algorithm parameter

3.7 Key Derivation

This implementation supports the following Key Derivation Functions (KDF):

Key Derivation Functions
KDF_NIST_800-108 KDF in Counter Mode with SHA-256 as PRF as specified in [5]
NIST.FIPS.186-4 Key Pair Generation Using Extra Random Bits with KDF in Counter Mode as RBG as specified in [6]
KDF NIST 800-56A One Pass C1E1S with “Single Step KDF” with HASH Sha256 as specified in [7]
ISO 15118 certificate installation and update as specified in [8]

Table 3-12 Key Derivation Functions

The KDF to use is specified by the algorithm key element (CRYPTO_KE_KEYDERIVATION_ALGORITHM) of the used parent key.
The following values can be configured:

ID	Name
1	CRYPTO_30_LIBCV_KDF_ALGO_KDF_SYM_NIST_800_108_CNT_MODE_SHA256
2	CRYPTO_30_LIBCV_KDF_ALGO_KDF_ASYM_NIST_FIPS_186_4_ERB
3	CRYPTO_30_LIBCV_KDF_ALGO_KDF_NIST_800_56_A_ONE_PASS_C1E1S_SINGLE_STEP_KDF_SHA256
4	CRYPTO_30_LIBCV_KDF_ALGO_KDF_ISO_15118_CERTIFICATE_HANDLING

Table 3-13 Values for Key Derivation Algorithm Key Element

3.7.1 Symmetric and Asymmetric Key Derivation

The behavior of the key derivation algorithm is configured by further specific key elements. These elements are
a password (CRYPTO_KE_KEYDERIVATION_PASSWORD),
a salt (CRYPTO_KE_KEYDERIVATION_SALT),
and the custom element label (CRYPTO_KE_CUSTOM_KEYDERIVATION_LABEL).

The number of iterations is automatically determined by the needed key length. The derived key is stored in the key element with id 1 of the given key.

3.7.2 Concatenation KDF

The key derive API is used to perform the concatenation KDF described in NIST 800-56A.

3.7.2.1 Preconditions

Before calling the API, the following key elements have to be set and must be marked as valid:

- **CRYPTO_KE_KEYDERIVATION_PASSWORD**
  This key element contains the private key.
- **CRYPTO_KE_CUSTOM_KEYDERIVATION_ADDITIONAL_INFO**
  This key element consists of the “Other Information” required for the algorithm.
- **CRYPTO_KE_CUSTOM_KEYEXCHANGE_PARTNER_PUB_KEY**
  This key element contains the public key needed for ECDH.

3.7.3 Smart Charge Communication (SCC)

The key derive API is used to perform the key installation/update described in ISO15118.

3.7.3.1 Preconditions

Before calling the API, the following key elements have to be set and must be marked as valid:

- **CRYPTO_KE_CUSTOM_SCC_CONTRACT_PUBLIC_KEY**
  This key element is the new public part of the encrypted new private key and consists of the concatenation of the two points.
  pubKey(64 bytes) = X(32 bytes) | Y(32 bytes)
- **CRYPTO_KE_CUSTOM_SCC_IV_AND_ENCRYPTED_PRIVATE_KEY**
  This key element consists of the concatenation of the IV and the encrypted new private key.
  Password(48 bytes) = IV(16 bytes) | ENC_PRIV(32 bytes)
- **CRYPTO_KE_CUSTOM_KEYEXCHANGE_PARTNER_PUB_KEY**
  This key element contains the public key needed for ECDH.

3.7.3.2 Certificate Installation

A provisioning key must be present at the key identified by cryptoKeyId. This can be achieved by setting the key element CRYPTO_KE_KEYDERIVATION_PASSWORD of cryptoKeyId with a key pair with KeyElementSet(). The targetCryptoKeyId specifies the destination for the key which shall be installed.

3.7.3.3 Certificate Update

A contract key must be present at the key identified by cryptoKeyId. This can be achieved by doing the certificate installation. The targetCryptoKeyId and cryptoKeyId are the same during certificate update.

3.8 Key Exchange

The implementation supports the following Key Exchange Algorithm (KEA):

Key Exchange Algorithm
ECDHE with ANSIp256r1
ECDHE with SECp256r1
ECDHE with X25519

Table 3-14 Key Exchange Algorithm

The public Key for ANSIp256r1 is stored a special format like described in Table 3-20.

3.9 Key Usage

To simplify the key usage this chapter describes some basic aspects.

3.9.1 Custom Key Elements

There are custom key elements defined, which are used to fulfill the algorithm requirements. These custom elements are defined in the Crypto_30_LibCv_Custom.h which need to be included in the Csm, e.g. via "`CsmCustomIncludeFiles`".

Custom Key Element	Value
CRYPTO_KE_CUSTOM_MAC_AES_ROUNDKEY	129u
CRYPTO_KE_CUSTOM_KEYDERIVATION_LABEL	130u
CRYPTO_KE_CUSTOM_KEYDERIVATION_ADDITIONAL_INFO	131u
CRYPTO_KE_CUSTOM_RSA_MODULUS	160u
CRYPTO_KE_CUSTOM_RSA_PUBLIC_EXPONENT	161u
CRYPTO_KE_CUSTOM_RSA_PRIVATE_EXPONENT	162u
CRYPTO_KE_CUSTOM_RSA_SALT	163u
CRYPTO_KE_CUSTOM_RSA_SALT_LENGTH	164u
CRYPTO_KE_CUSTOM_KEYEXCHANGE_PARTNER_PUB_KEY	3003u
CRYPTO_KE_CUSTOM_SCC_CONTRACT_PUBLIC_KEY	3013u
CRYPTO_KE_CUSTOM_SCC_IV_AND_ENCRYPTED_PRIVATE_KEY	3014u
CRYPTO_KE_CUSTOM_RANDOM_PERSONALIZATION_STRING	3015u
CRYPTO_KE_CUSTOM_RANDOM_ADDITIONAL_INPUT	3016u
CRYPTO_KE_CUSTOM_RANDOM_NONCE	3017u
CRYPTO_KE_CUSTOM_RANDOM_RESEED_COUNTER	3018u
CRYPTO_KE_CUSTOM_SHE_COUNTER	3019u
CRYPTO_KE_CUSTOM_SHE_UID	3021u
CRYPTO_KE_CUSTOM_RSA_PRIME_P	3051u
CRYPTO_KE_CUSTOM_RSA_PRIME_Q	3052u
CRYPTO_KE_CUSTOM_RSA_EXPONENT_DP	3053u
CRYPTO_KE_CUSTOM_RSA_EXPONENT_DQ	3054u
CRYPTO_KE_CUSTOM_RSA_INVERSE_QI	3055u
CRYPTO_KE_CUSTOM_SHE_BOOT_PROTECTION	3056u
CRYPTO_KE_CUSTOM_SHE_DEBUGGER_PROTECTION	3057u

Table 3-15 Custom Key Elements

3.9.2 Key Element Access Rights

The CRYPTO driver provides the following key access rights with the shown security strength.

Access Right	Value	Access Strength	Description
ALLOWED	0	Lowermost	Accessible by plaintext.
ENCRYPTED	1		Encrypted external access only.
INTERNAL_COPY	2		No external access, only internal access.
DENIED	3	Highest	No external access, only internal job usage.

Table 3-16 Key Element Access Rights Ranking



Expert Knowledge
The read access right of key elements should be stricter or equal to the write access right to avoid secret keys easily read in plaintext.

The key element access rights will be checked according the following table.

Access	Key	Read Access Right	Write Access Right	Recommended Setting
KeyElementGet	Key	RA_ALLOWED	-	
GET while ProcessJob	Key	ALL	-	
KeyElementSet	Key	-	WA_ALLOWED	
	SHE Key	ALL	WA_ENCRYPTED	RA_DENIED
KeyElementCopy* KeyCopy*	Source	RA right <= RA_INTERNAL_COPY	-	
	Destination	RA right >= Source Key	WA right <= WA_INTERNAL_COPY	
KeyDerive	Source	ALL	-	
	Destination	-	WA right <= WA_INTERNAL_COPY	
KeyExchange	Own Public	ALL	WA right <= WA_INTERNAL_COPY	RA_ALLOWED
	Private	ALL	WA right <= WA_INTERNAL_COPY	
	Shared	ALL	WA right <= WA_INTERNAL_COPY	
RandomSeed	Seed FIPS	-	WA_ALLOWED	
	Seed DRBG	RA_DENIED	WA_ALLOWED	
	ReseedCnt	ALL	WA_DENIED	

- : no effect, not used in function
ALL : key is accesed in function but all access rights are accepted
* : SHE keys cannot be copied

Table 3-17 Key Element Access Rights

3.9.3 Key Locking

The key is locked during the complete processing to hold the key and its key elements consistent. To allow a high performance it is possible to lock a key from several jobs for reading. A key can only be written if there is neither a read nor a write lock yet. If there is a write lock the key cannot be read.

3.9.4 Pre-Configured Key Types

The CRYPTO provides a range of pre-configured key types. These pre-configured key types shall provide the most recommended usage scenarios. Further key types may be added by the user. Preconfigured key types could be used as template by duplicating them.

Not used key types will be skipped by the generator and have therefore no negative influence on the code and ram size.



Expert Knowledge
To reduce the Ram usage configure the key type only with the required key elements. Also configure the key elements only with the required size.

3.9.5 Key Validity

The validity of the key is checked and modified by the service function according to the following table. If the key element has an init value the key element is set valid during the initialization excluding SHE keys.

Validity / Service	read key need to be valid	modified key is valid afterwards	modified key is invalid afterwards
ProcessJob	X		
KeyElementGet	X		
KeyElementSet	X*		X
KeyElementCopy			X
KeyCopy			X
KeyValidSet		X	
KeyDerive	X	X	
KeyExchangeCalcSecret	X		X
KeyExchangeCalcPubVal	X		X
RandomSeed	X	X	
X* SHE key update			

Table 3-18 Key Validity

3.9.6 Key Element Delete

The Crypto supports a feature to delete a key element. To do this, the service Crypto_30_LibCv_KeyElementSet need to be called with a valid keyPtr and the keyLength 0. The DET check for the keyLength will be disabled if the feature is enabled. Key element delete is not supported for virtual key elements. If the key is set valid and read out the CRYPTO will return a key element with the length 0, which is not accepted by the most primitives.



Note
The key element will be overwritten once with zeros.

3.10 Algorithm

The algorithm specific handling is described in this chapter.

3.10.1 AES CMAC Round-key Reuse

The AES CMAC supports the reuse of calculated AES round keys. If this feature is enabled the round keys are hold in the driver object. The round keys are hold until a different job type is processed (not CMAC AES128) or the corresponding key has changed.



Expert Knowledge
To ensure that the max speedup configure a driver object for each AES CAMC key. The driver object can be used for several AES CMAC jobs with the same key.

3.10.2 RSA

The keys which are required by the RSA algorithm are stored in separate key elements which are defined in Table 3-15. The RSA PSS requires a salt for calculation which is always required. For the signature verification only, the salt length is required. If the salt length is not given or shall be calculated automatically the key element CRYPTO_KE_CUSTOM_RSA_SALT_LENGTH must be excluded from the used key.

Key Element / RSA Primitive	RSA_MODULUS	PUBLIC_EXPONENT	PRIVATE_EXPONENT	RSA_SALT	RSA_SALT_LENGTH	RSA_PRIME_P	RSA_PRIME_Q	RSA_EXPONENT_DP	RSA_EXPONENT_DQ	RSA_INVERSE_QI
Signature Generate PKCS#1 V1.5	X		X							
Signature Verify PKCS#1 V1.5	X	X								
Signature Generate PKCS#1 V1.5 CRT						X	X	X	X	X
Signature Generate PKCS#1 V2.2 PSS	X		X	X						
Signature Verify PKCS#1 V2.2 PSS	X	X			X					
Encrypt PKCS#1 V1.5	X	X								
Decrypt PKCS#1 V1.5	X		X							

Table 3-19 RSA Key Element Usage

3.10.3 ECDSA

The ECDSA requires a public key x and a public key y for verification. The public key x and public key y are stored in the signature key element. Since there is only one key element used, the key x and key y need to be stored concatenated. The size of the public key x and the public key y are the same so that the signature key element size is the double.

Signature Key Element (Size 2L)	
public key x (Size L)	public key y (Size L)

Table 3-20 ECDSA Signature Key Element Storage

The signature for ECDSA is split in signature R and signature S. The signature is saved in the job output buffer. To do this, the signature R and signature S are concatenated. Signature R and signature S have the same size.

Signature Buffer	
Signature R	Signature S

Table 3-21 ECDSA Signature Storage

3.10.4 RNG FIPS

The random number generator is implemented according to [9].The FIPS 186-2 can only have one instance per driver object. The internal state of the RNG is stored in the driver object. The service RandomSeed is used to set the new entropy. If a new entropy is available the RNG is seeded when the generation job is processed. During the random number generation, the key is locked for writing.



Caution

If one key is used for several RNG jobs, only the first executed random instance gets seeded with this entropy. Therefore, it is highly recommended that each random instance has its own key, to avoid side effects.

3.10.5 RNG DRBG

The random number generator is implemented according to [10]. The chosen variant is DRBG-CTR AES 128 with DF and without DF. The RNG does not provide the features according to [10] for prediction resistance check and self / health test function. If all parameters are set corresponding to the requirements in [10] the implementation obtain a security strength of 128.

Due the implementation for embedded hardware all buffers and input parameter are limited to a length of the max value of uint32. The max output bit length is limited to 1024Bytes. The Temp variable for the generated random number is not used, the output is written directly to the given outputPtr to reduce stack usage.

The DRBG instance save all required values in the key. Thereby it is possible to have multiple instances of the DRBG in one driver object. It is not possible to configure DRBG-

CTR AES 128 with DF and without DF at the same time. During the random number generation, the key is locked for writing.

The key elements CRYPTO_KE_CUSTOM_RANDOM_RESEED_COUNTER and CRYPTO_KE_RANDOM_SEED_STATE need to be persisted to safe the internal state of the DRBG. The DRBG get instantiated with the first Crypto_30_libCv_RandomSeed call. Each further call will be a reseeding. To uninstantiate, the key element CRYPTO_KE_RANDOM_SEED_STATE ne to be deleted via Crypto_30_LibCv_KeyElementSet. Afterwards the DRBG instance can be instantiated again.

The DRBG requires different input parameter for its operations. The required key elements are shown in the Table 3-22. The optional and recommended key elements must be excluded from the configured key if the input is not necessary.

DRBG Operation		Key Element: RANDOM_SEED_STATE	RANDOM_RESEED_COUNTER	RANDOM_ADDITIONAL_INPUT	RANDOM_PERSONALIZATION_STRING	RANDOM_NONCE
Without DF	Instantiation	X	X		R	
	Reseed	X	X	O		
	Generate Random Number	X	X	O		
With DF	Instantiation	X	X		R	X
	Reseed	X	X	O		
	Generate Random Number	X	X	O		

X: required
R: Recommended
O: Optional

Table 3-22 DRBG required key elements



Expert Knowledge
Use only one DRBG instance and reseed this instance as often as possible with new entropy to increase the numbers randomness. The entropy source is an important part to increase the numbers randomness.

3.10.6 AES Encryption/Decryption

The AES encryption/decryption supports key length of 128 and 256 bit. If a AES job is executed the CRYPTO distinguishes the AES variant on the given key length.



Enabling AES 256 support
The support of AES 256 needs to be activated in the general settings of the module configuration.

3.11 Driver Objects and Jobs

The CRYPTO driver supports multiple driver objects. Each driver object has its own workspace. Algorithms share this workspace so that the size of the workspace depends on the configured algorithm with the highest RAM usage (e.g. RSA).

As soon as a job is started on a driver object for one algorithm, driver object is locked and therefore busy for another algorithm. If it is necessary to execute another job in between, a further driver object can be created and used for this job. As mentioned above, each driver object has its own workspace and can therefore be used to execute jobs parallel.



Unique Jobs
Jobs should only be owned and called by only one application since the driver cannot differ between one job from different callers. Ensure that a job is only called once at a time.

3.12 Memory Access

3.12.1 NVM Support

The CRYPTO supports to persist key elements in the NVM Therefore the reference to a NVM block must be configured. The validation of the DaVinci Configurator 5 configures the block automatically to the needs of the CRYPTO.



Expert Knowledge
The configured length of the corresponding NVM block can differ from the real length of the Crypto_30_KeyStorage variable. This is because the variable keeps also track of all RAM key elements which are in the uppermost bytes of the storage.

The length is determined by the validation and can also be seen in the generated define CRYPTO_30_LIBCV_PERSIST_SIZE.

To ensure the correct restoring of the persisted key keep the initialization in the correct order:

- Crypto_30_LibCv_InitMemory (INIT_MEMORY)
- Crypto_30_LibCv_Init (INIT_TWO_DRV)
- NvM_ReadAll

The CRYPTO module does not handle its configured NVM block explicitly. Therefore, the NVM block needs to be read manually or during the NvM_ReadAll operation. For persisting keys, the block needs either be written manually or needs to be included in the NvM_WriteAll operation.

The CRYPTO provides an optional feature to call NvM_SetRamBlockStatus whenever a key which need to be persist is validated. If a key is validated can be check in Table 3-18. If the feature is disabled the CRYPTO module does not mark a block as modified (NvM_SetRamBlockStatus), it's up to the NVM to detect the need of writing the block.

3.12.2 vStdLib

The CRYPTO module can use the vStdLib to increase extensive memory writings.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR CRYPTO into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the CRYPTO contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
Crypto_30_LibCv.c	This is the main source file of the CRYPTO
Crypto_30_LibCv.h	This is the source file of the CRYPTO
Crypto_30_LibCv_Aead.c	This source contains AEAD algorithms
Crypto_30_LibCv_Cipher.c	This source contains cipher algorithms
Crypto_30_LibCv_Hash.c	This source contains HASH algorithms
Crypto_30_LibCv_KeyManagement.c	This source contains the CRYPTO´s key management functions
Crypto_30_LibCv_KeyManagement.h	This header contains the CRYPTO´s key management functions
Crypto_30_LibCv_Mac.c	This source contains MAC algorithms
Crypto_30_LibCv_Random.c	This source contains PRNG algorithms
Crypto_30_LibCv_Services.h	This header file is used for the internal dispatcher
Crypto_30_LibCv_Signature.c	This source contains signature algorithms
Crypto_30_LibCv_Custom.h	This header contains internal defines.
Crypto_30_LibCv_Curve.h	This header contains internal elliptic curve parameter declaration.
Crypto_30_LibCv_Curve_Int.h	This header contains internal elliptic curve parameter definition.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator 5 Pro.

File Name	Description
Crypto_30_LibCv_Cfg.c	This is configuration source file.
Crypto_30_LibCv_Cfg.h	This is configuration header file.

Table 4-2 Generated files

© 2018 Vector Informatik GmbH

4.2 Critical Sections

Crypto uses the following critical sections:

> CRYPTO_30_LIBCV_EXCLUSIVE_AREA_0

This critical section protects workspace locking resources and so this section should never be interrupted by any other API of the Crypto module.

> CRYPTO_30_LIBCV_EXCLUSIVE_AREA_1

This critical section protects 32 Bit accesses and is only necessary in case of HW doesn't offer an atomic access.

4.3 Best Practice

This chapter offers some best practice to simplify the crypto stack configuration.

4.3.1 Bottom to Top

The best way to configure keys, key types and key elements is to start configuration at the CRYPTO driver before doing this steps in CryIf and Csm. Using this way, the DaVinci Configurator offers a solving action to propagate the keys from the CRYPTO driver to the upper layers.

This also applies to the CYPTO driver objects and CSM job assignment. First plan which CSM jobs shall be processed and check the calling conditions. Secondly create the CRYPTO driver objects with the CRYPO primitives to fulfill the conditions. Thirdly create the CSM Jobs and refence the CRYPTO driver objects.

5 API Description

For an interfaces overview please see Figure 2-2.

5.1 Services provided by CRYPTO

5.1.1 Crypto_30_LibCv_Init

Prototype	
`void` **`Crypto_30_LibCv_Init`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes the Crypto Driver. This function initializes the module Crypto_30_LibCv. It initializes all variables and sets the module state to initialized.	
Particularities and Limitations	
Specification of module initialization > Interrupts are disabled. Module is uninitialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-1 Crypto_30_LibCv_Init

5.1.2 Crypto_30_LibCv_InitMemory

Prototype	
`void` **`Crypto_30_LibCv_InitMemory`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
The function initializes variables, which cannot be initialized with the startup code. Initializes component variables at power up.	
Particularities and Limitations	
Module is uninitialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-2 Crypto_30_LibCv_InitMemory

5.1.3 Crypto_30_LibCv_MainFunction

Prototype	
`void` **`Crypto_30_LibCv_MainFunction`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Main function of the module. Is called cyclically and handles asynchronous jobs. This interface process the provided asynchronous jobs from the queue. This interface process the provided asynchronous jobs from the queue.	
Particularities and Limitations	
Module is uninitialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-3 Crypto_30_LibCv_MainFunction

5.1.4 Crypto_30_LibCv_GetVersionInfo

Prototype	
`void` **`Crypto_30_LibCv_GetVersionInfo`** `(Std_VersionInfoType *versioninfo)`	
Parameter	
versioninfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information. Crypto_30_LibCv_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Particularities and Limitations	
Configuration Variant(s): CRYPTO_30_LIBCV_VERSION_INFO_API == STD_ON	
Call context	
> TASK|ISR > This function is Synchronous > This function is Reentrant	

Table 5-4 Crypto_30_LibCv_GetVersionInfo

5.1.5 Crypto_30_LibCv_ProcessJob

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_ProcessJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in,out]	Pointer to the configuration of the job. Contains structures with job and primitive relevant information but also pointer to result buffers.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_NOT_VALID Request failed, the key is not valid.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, a key element has the wrong size.
	CRYPTO_E_QUEUE_FULL Request failed, the queue is full.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.

	CRYPTO_E_ENTROPY_EXHAUSTION Request failed, the entropy is exhausted
	CRYPTO_E_JOB_CANCELED The service request failed because the synchronous job has been canceled.
Functional Description	
Process the received job. Performs the crypto primitive that is configured in the job parameter.	
Particularities and Limitations	
none	
Call context	
> TASK > This function is Reentrant	

Table 5-5 Crypto_30_LibCv_ProcessJob

5.1.6 Crypto_30_LibCv_CancelJob

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_CancelJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in,out]	Pointer to the configuration of the job. Contains structures with user and primitive relevant information.
Return code	
Std_ReturnType	E_OK Request successful, job has been removed.
Std_ReturnType	E_NOT_OK Request failed, job could not be removed.
Functional Description	
Cancels the received job. This interface removes the provided job from the queue and cancels the processing of the job if possible.	
Particularities and Limitations	
none	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-6 Crypto_30_LibCv_CancelJob

5.2 Key Management Functions

5.2.1 Crypto_30_LibCv_KeyCopy

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyCopy`** `(uint32 cryptoKeyId, uint32 targetCryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
targetCryptoKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.
Functional Description	
Copy the key. Copies a key with all its elements to another key in the same crypto driver. The key element can only be copied, if the destination key element write access right is less than WA_INTERNAL_COPY. Additional, the read access right of the source must be less than RA_INTERNAL_COPY and the destination read access right must be higher or equal than the source read access right.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-7 Crypto_30_LibCv_KeyCopy

5.2.2 Crypto_30_LibCv_KeyElementCopy

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyElementCopy`** `(uint32 cryptoKeyId, uint32 keyElementId, uint32 targetCryptoKeyId, uint32 targetKeyElementId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
keyElementId [in]	Holds the identifier of the key element which shall be the source for the copy operation.
targetCryptoKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
targetKeyElementId [in]	Holds the identifier of the key element which shall be the destination for the copy operation.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_EXTRACT_DENIED Request failed, not allowed to extract key material.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.
Functional Description	
Copy key element. Copies a key element to another key element in the same crypto driver. The key element can only be copied, if the destination key element write access right is less than WA_INTERNAL_COPY. Additional the read access right of the source must be less than RA_INTERNAL_COPY and the destination read access right must be higher or equal than the source read access right.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-8 Crypto_30_LibCv_KeyElementCopy

5.2.3 Crypto_30_LibCv_KeyElementIdsGet

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyElementIdsGet`** `(uint32 cryptoKeyId, uint32 *keyElementIdsPtr, uint32 *keyElementIdsLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose available element ids shall be exported.
keyElementIdsLengthPtr [in]	Holds a pointer to the memory location in which the number of key element in the given key is stored. On calling this function, this parameter shall contain the size of the buffer provided by keyElementIdsPtr. When the request has finished, the actual number of key elements is stored.
keyElementIdsPtr [out]	Contains the pointer to the array where the ids of the key elements shall be stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Used to retrieve information which key elements are available in a given key. The service provides a list of key elements which are available in the given key.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-9 Crypto_30_LibCv_KeyElementIdsGet

5.2.4 Crypto_30_LibCv_KeyElementSet

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyElementSet`** `(uint32 cryptoKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
keyPtr [in]	Holds the pointer to the key data which shall be set as key element.
keyLength [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element size does not match size of provided data.
Functional Description	
Sets a key element. Sets the given key element bytes to the key identified by cryptoKeyId. The key element can only be set, if the write access right is WA_ALLOWED or WA_ENCRYPTED for she key update.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-10 Crypto_30_LibCv_KeyElementSet

5.2.5 Crypto_30_LibCv_KeyValidSet

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyValidSet`** `(uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key elements shall be set to valid.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Sets the key to valid. Sets the key state of the key identified by cryptoKeyId to valid.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-11 Crypto_30_LibCv_KeyValidSet

5.2.6 Crypto_30_LibCv_KeyElementGet

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyElementGet`** `(uint32 cryptoKeyId, uint32 keyElementId, uint8 *resultPtr, uint32 *resultLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
resultPtr [in]	Holds the pointer to the key data which shall be set as key element.
resultLengthPtr [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the provided buffer is too small to store the result.
Functional Description	
This interface shall be used to get a key element of the key identified by the cryptoKeyId and store the key element in the memory location pointed by the result pointer. The key element can only be provided, if the read access right is RA_ALLOWED.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-12 Crypto_30_LibCv_KeyElementGet

5.2.7 Crypto_30_LibCv_RandomSeed

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_RandomSeed`** `(uint32 cryptoKeyId, const uint8 *entropyPtr, uint32 entropyLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key for which a new seed shall be generated.
entropyPtr [in]	Holds a pointer to the memory location which contains the data to feed the entropy.
entropyLength [in]	Contains the length of the entropy in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Initializes the seed. This function generates the internal seed state using the provided entropy source. Furthermore, this function can be used to update the seed state with new entropy.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-13 Crypto_30_LibCv_RandomSeed

5.2.8 Crypto_30_LibCv_KeyGenerate

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyGenerate`** `(uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which is to be updated with the generated value.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Generates a key. This function shall dispatch the key generate function to the configured crypto driver object.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-14 Crypto_30_LibCv_KeyGenerate

5.2.9 Crypto_30_LibCv_KeyDerive

Prototype	
`Std_ReturnType **Crypto_30_LibCv_KeyDerive** (uint32 cryptoKeyId, uint32 targetCryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which is used for key derivation.
targetCryptoKeyId [in]	Holds the identifier of the key which is used to store the derived key.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Derives a key. Derives a new key by using the key elements in the given key identified by the cryptoKeyId. The key derivation function supports different algorithm which are specified by the algorithm key element(CRYPTO_KE_KEYDERIVATION_ALGORITHM). For CRYPTO_30_LIBCV_KDF_ALGO_KDF_SYM_NIST_800_108_CNT_MODE_SHA256 and CRYPTO_30_LIBCV_KDF_ALGO_KDF_ASYM_NIST_FIPS_186_4_ERB: The given key must contain the key elements for the password (CRYPTO_KE_KEYDERIVATION_PASSWORD), salt (CRYPTO_KE_KEYDERIVATION_SALT), algorithm (CRYPTO_KE_KEYDERIVATION_ALGORITHM) and the custom element label(CRYPTO_KE_CUSTOM_KEYDERIVATION_LABEL). The number of iterations is automatically determined by the needed key length. The derived key is stored in the key element with id 1 of the given key identified by targetCryptoKeyId. The write access right for the destination key element must be less or equal than WA_INTERNAL_COPY. For CRYPTO_30_LIBCV_KDF_ALGO_KDF_NIST_800_56_A_ONE_PASS_C1E1S_SINGLE_STEP_KDF_SHA256: The KDF is reads the algorithm configuration from the cryptoKeyId key elements. The KDF requires the following key elements the private key (CRYPTO_KE_KEYDERIVATION_PASSWORD), the public key (CRYPTO_KE_CUSTOM_KEYEXCHANGE_PARTNER_PUB_KEY) and the other Information (CRYPTO_KE_CUSTOM_KEYDERIVATION_ADDITIONAL_INFO). The derived key is set at the target key element (CRYPTO_KE_KEYDERIVATION_PASSWORD). For CRYPTO_30_LIBCV_KDF_ALGO_KDF_ISO_15118_CERTIFICATE_HANDLING: For certificate installation the cryptoKeyId needs to be the root certificate. For certificate update the cryptoKeyId and targetCryptoKeyId need to be the same key Id. The KDF reads the algorithm configuration from the cryptoKeyId key elements. The KDF requires the private key as key element (CRYPTO_KE_KEYDERIVATION_PASSWORD). The following key elements need to be set at the targetCryptoKeyId: the contract/new public key (CRYPTO_KE_CUSTOM_SCC_CONTRACT_PUBLIC_KEY), the IV and encrypted private key (CRYPTO_KE_CUSTOM_SCC_IV_AND_ENCRYPTED_PRIVATE_KEY) and the public key (CRYPTO_KE_CUSTOM_KEYEXCHANGE_PARTNER_PUB_KEY). The derived key is set at the target key element (CRYPTO_KE_KEYDERIVATION_PASSWORD).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-15 Crypto_30_LibCv_KeyDerive

5.2.10 Crypto_30_LibCv_KeyExchangeCalcPubVal

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyExchangeCalcPubVal`** `(uint32 cryptoKeyId, uint8 *publicValuePtr, uint32 *publicValueLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which shall be used for the key exchange protocol.
publicValuePtr [out]	Contains the pointer to the data where the public value shall be stored.
publicValueLengthPtr [in,out]	Holds a pointer to the memory location in which the public value length information is stored. On calling this function, this parameter shall contain the size of the buffer provided by publicValuePtr. When the request has finished, the actual length of the returned value shall be stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Calculation of the public value. Calculates the public value for the key exchange and stores the public key in the memory location pointed by the public value pointer. The write access right for the destination key element must be less or equal than WA_INTERNAL_COPY.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-16 Crypto_30_LibCv_KeyExchangeCalcPubVal

5.2.11 Crypto_30_LibCv_KeyExchangeCalcSecret

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_KeyExchangeCalcSecret`** `(uint32 cryptoKeyId, const uint8 *partnerPublicValuePtr, uint32 partnerPublicValueLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which shall be used for the key exchange protocol.
partnerPublicValuePtr [in]	Holds the pointer to the memory location which contains the partners public value.
partnerPublicValueLength [in]	Contains the length of the partners public value in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Calculation of the secret. Calculates the shared secret key for the key exchange with the key material of the key identified by the cryptoKeyId and the partner public key. The shared secret key is stored as a key element in the same key. The write access right for the destination key element must be less or equal than WA_INTERNAL_COPY.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-17 Crypto_30_LibCv_KeyExchangeCalcSecret

5.2.12 Crypto_30_LibCv_CertificateParse

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_CertificateParse`** `(uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key slot in which the certificate has been stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Parse stored certificate. Parses the certificate data stored in the key element CRYPTO_KE_CERT_DATA and fills the key elements CRYPTO_KE_CERT_SIGNEDDATA, CRYPTO_KE_CERT_PARSEDPUBLICKEY and CRYPTO_KE_CERT_SIGNATURE	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-18 Crypto_30_LibCv_CertificateParse

5.2.13 Crypto_30_LibCv_CertificateVerify

Prototype	
`Std_ReturnType` **`Crypto_30_LibCv_CertificateVerify`** `(uint32 cryptoKeyId, uint32 verifyCryptoKeyId, Crypto_VerifyResultType *verifyPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which shall be used to validate the certificate.
verifyCryptoKeyId [in]	Holds the identifier of the key containing the certificate, which shall be verified.
verifyPtr [out]	Holds a pointer to the memory location which will contain the result of the certificate verification.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Certificate verification. Verifies the certificate stored in the key referenced by verifyCryptoKeyId with the certificate stored in the key referenced by cryptoKeyId.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-19 Crypto_30_LibCv_CertificateVerify

5.3 Services needed by CRYPTO

5.3.1 esl_getBytesRNG

Prototype	
`eslt_ErrorCode` **`esl_getBytesRNG`** `(actU16 target_length, actU8* target)`	
Parameter	
target_length [in]	Holds the length of random bytes needed
target [in/out]	Holds the pointer to the location where the random bytes shall be stored. This buffer has to be at least ‘target_length’
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request successful.
	ESL_ERC_ERROR Request failed.
Functional Description	
The esl_getBytesRNG is needed for elliptic curve algorithms to get pseudo randomly generated bytes. Currently there is no way to get entropy in the SecMod library so this function has to be implemented manually by the customer. This function shall provide ‘target_length’ amount of random bytes and write the output to the buffer given by ‘target’ pointer.	
Particularities and Limitations	
none	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-20 esl_getBytesRNG



Caution
Please include “ESLib.h” for the correct type definitions.

5.4 Services used by CRYPTO

In the following table services provided by other components, which are used by the CRYPTO are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
CryIf	CryIf_CallbackNotification
NvM	NvM_SetRamBlockStatus
VStdLib	VStdLib_MemCpy VStdLib_MemClr

Table 5-21 Services used by the CRYPTO

6 Configuration

In the CRYPTO the attributes can be configured according to/ with the following methods/ tools:

- Configuration in DaVinci Configurator 5 Pro

6.1 Configuration Variants

The CRYPTO supports the configuration variants

- VARIANT-PRE-COMPILE

The configuration classes of the CRYPTO parameters depend on the supported configuration variants. For their definitions please see the Crypto_30_LibCv_bswmd.arxml file.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
CSM	Crypto Service Manager
CRYIF	Crypto Interface
CRYPTO	Crypto Driver

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
AES	Advanced Encryption Standard
ALGOFAM	Algorithm Family
ALGOMODE	Algorithm Mode
ANSI	American National Standards Institute
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CBC	Cipher Block Chaining
CMAC	Cipher-based Message Authentication Code
CRT	Chinese Remainder Theorem
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DF	Derivation Function
EAD	Embedded Architecture Designer
ECB	Electronic Code Book
ECC	Elliptic-curve-cryptography
ECDHE	Elliptic-curve Diffie–Hellman
ECDSA	Elliptic-curve Digital Signature Algorithm
ECU	Electronic Control Unit
FIPS	Federal Information Processing Standards Publication
GCM	Galois Counter Mode
HIS	Hersteller Initiative Software
HMAC	Keyed-Hash Message Authentication Code
ISO	International Organization for Standardization
ISR	Interrupt Service Routine
KDF	Key Derivation Function

Abbreviation	Description
KE	Key Element
KEA	Key Exchange Algorithm
MAC	Message Authentication Code
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
NIST	National Institute of Standards and Technology
PPORT	Provide Port
PRF	Pseudorandom Function
PRNG	Pseudo Random Number Generator
RNG	Random Number Generator
RPORT	Require Port
RSA	Rivest-Shamir-Adleman
RTE	Runtime Environment
SCC	Smart Charge Communication
SEC	Standards for Efficient Cryptography
SHA	Secure Hash Algorithm
SHE	Secure Hardware Extension
SipHash	short-input PRF Hash
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification
TRNG	True Random Number Generator
UID	Unique identification
FID	Functional identification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com